

Spring Security

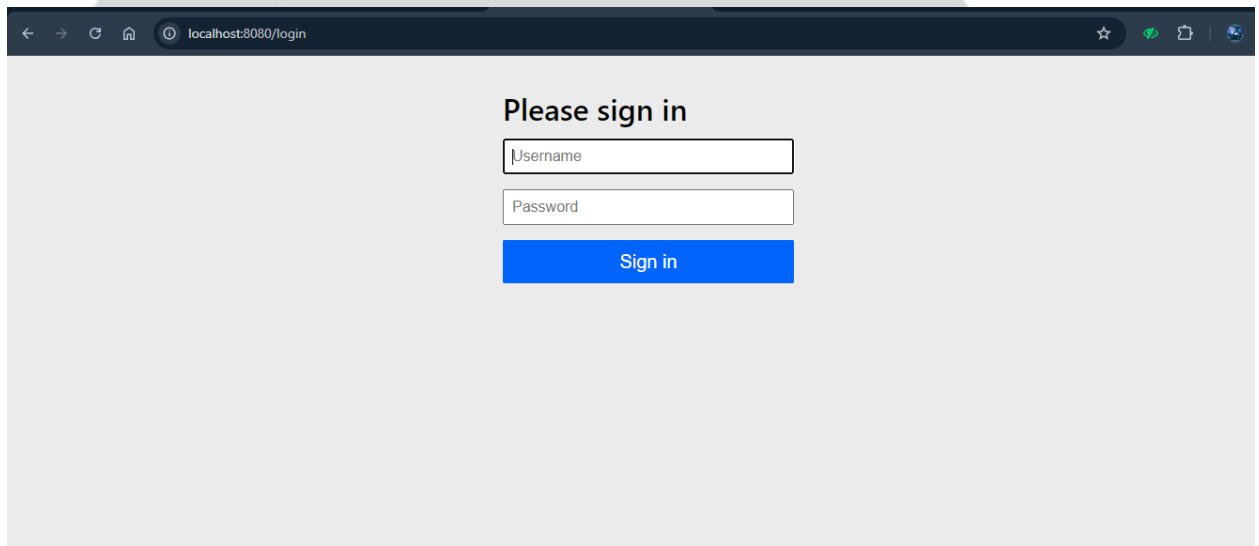
A quick Guide to learn spring security

For every application we need the security to make data secure. In spring boot we need to provide the security to the application using the spring security.

Creating a springboot security project :

1. Create a spring starter project
2. Add dependencies
 - a. Spring web
 - b. Spring dev tool (optional)
 - c. Database driver
 - d. Spring jpa
 - e. Spring security
3. Once it is done create a REST controller.
4. It will create a login form by default for us and we can find the password in the console. For user-name we have to use user.

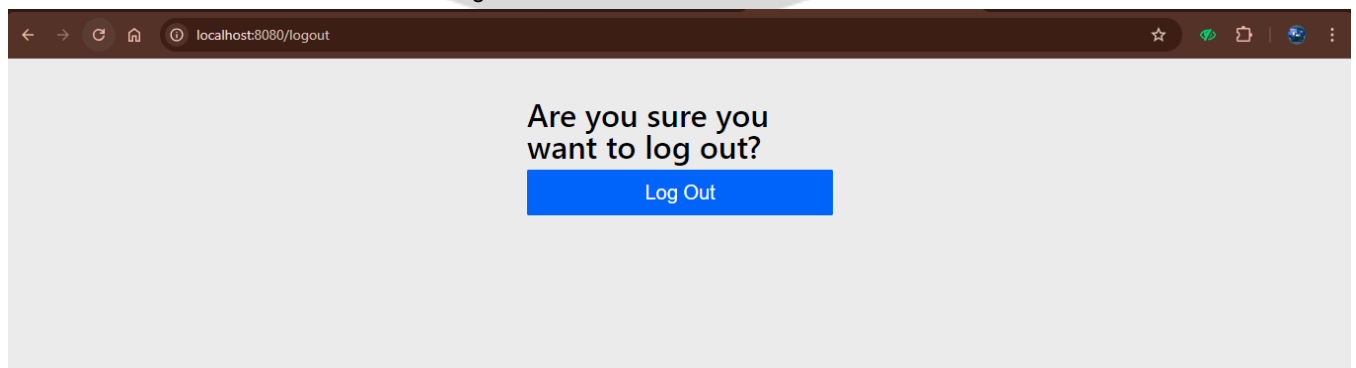
Example :



5. If you want to change the password we have to go to application.properties, we can only set for one user for the application.
6. If you want to logout from the application we can simply state as show in below syntax

Syntax :

localhost:8080/logout



Spring Security

A quick Guide to learn spring security

Working of spring Security :

1. When ever we are trying to get the response from the server and also to handle the request. These two will be done by the servlet container which handles it.
2. In general we will not include the spring in this scenario but we have to include it in real time application to provide the authorization and authentication.
3. To provide it spring security will focus on the filters to provide a proper flow of data, where it will transfer from controller to so on.
4. We can even customize the filters as well in spring security.

NOTE : Everytime when you re-login to the page it will create a new session id. That means different users will have different session id's.

Changing the username and password :

1. To change the username and password we have to configure the UsernamePasswordAuthenticationFilter filter.
2. To do that we have to enter the username and password in the property file.

Syntax :

```
spring.security.user.name = nameOfUser  
Spring.security.user.password = userPassword
```

2

3 `spring.security.user.name = root`

4 `spring.security.user.password = 0000`

3. When you want to login use the given super root username and password



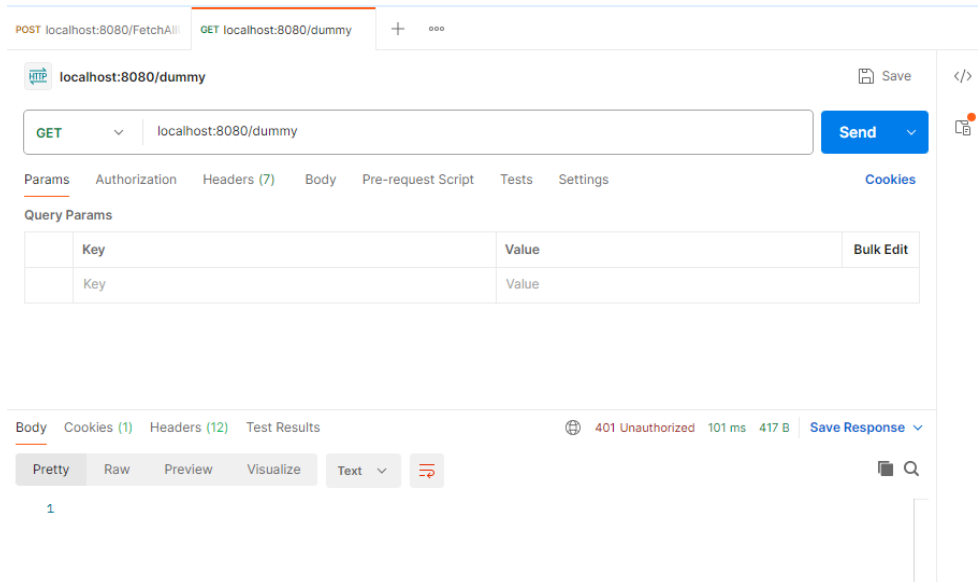
4. It works fine with the created user name and password.

Connecting with the postman :

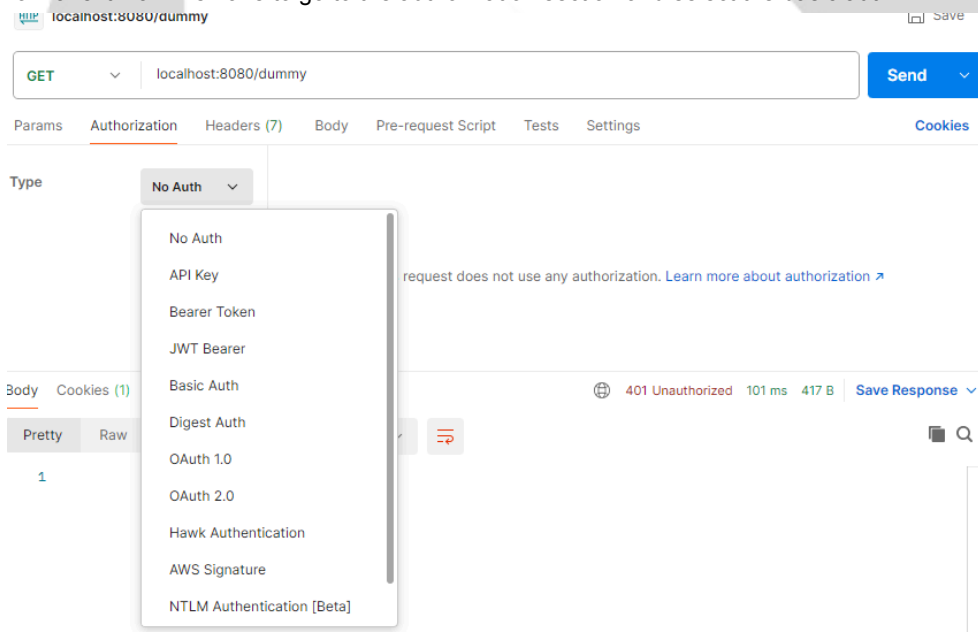
1. To connect with the post man we have provide the authorization to it or else it will reflect status code 401 not authorized to us.
2. To do it follow the below steps

Spring Security

A quick Guide to learn spring security



3. To make it work we have to go to the authorization section and select the basic auth



Spring Security

A quick Guide to learn spring security

4. Enter the username and password in the respective fields

The screenshot shows the Postman interface for a GET request to `localhost:8080/dummy`. The **Authorization** tab is selected, and the type is set to **Basic A...**. The **Username** field contains `root` and the **Password** field contains `0000`. The status bar at the bottom indicates a **401 Unauthorized** response with a status of `401`, a time of `101 ms`, and a size of `417 B`.

5. Then it will connect with the postman

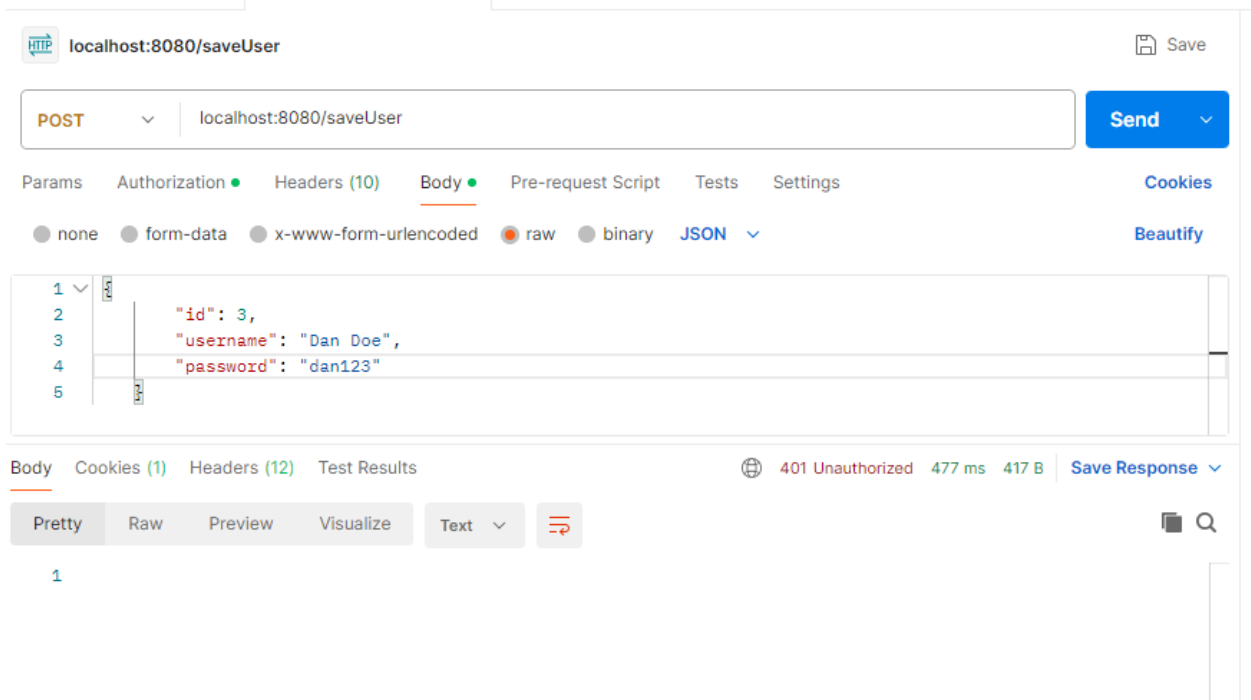
The screenshot shows the Postman interface for a GET request to `localhost:8080/dummy`. The **Authorization** tab is selected, and the type is set to **Basic A...**. The **Username** field contains `root` and the **Password** field contains `0000`. The status bar at the bottom indicates a **200 OK** response with a status of `200`, a time of `172 ms`, and a size of `390 B`. The **Body** tab is selected, showing the response text: `1 This is a dummy message 1994F80F89096A81D9EEAC588249ED3B`.

CSRF (Cross Site Request Frogery) Token:

1. CSRF is an attack where the attacker will send a unintended request from the client browser to a verified website.
2. For example imagine client is logged into the bank application and at the same time he visited an malicious website and visited a link which is having the CSRF token.
3. Then this malicious website will send a request to the bank server for a request as a user as he is already logged in to the bank and bank server will think that it is the user asking for the request and fulfill it.
4. Hence it will create a huge security problem as a developer we have to make sure it is not the case in the application.
5. By default spring asks us to handle the CSRF to use the POST, PUT and DELETE methods.

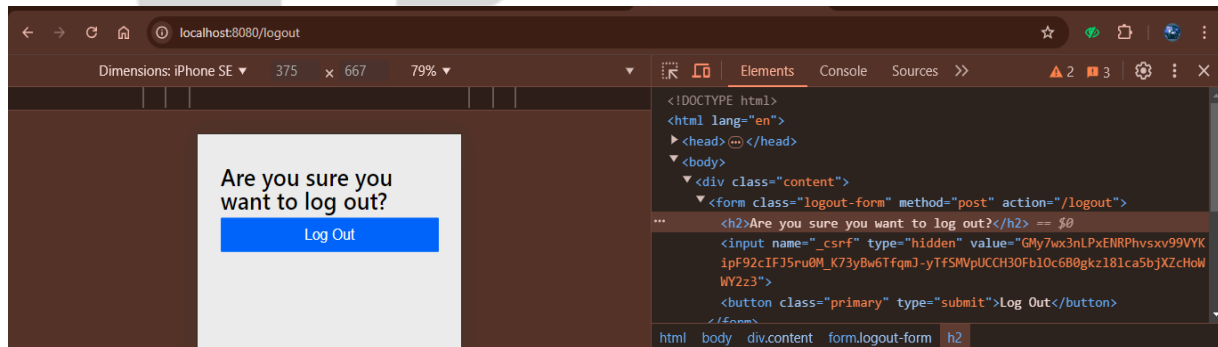
Spring Security

A quick Guide to learn spring security



- To handle the CSRF the server will generate a token whenever we are trying to save, update or delete the data.

NOTE : If you don't know the CSRF token just goto logout page and goto inspect in browser and there we can find the CSRF token value



- To generate the token value we have to request for the token to the server create a method to get the CSRF token.

Syntax :

```
@GetMapping("/getCSRFToken")
Public CsrfToken getCsrfToken(HttpServletRequest request){
    Return (CSRF) request.getAttribute("csrfName");
}
```

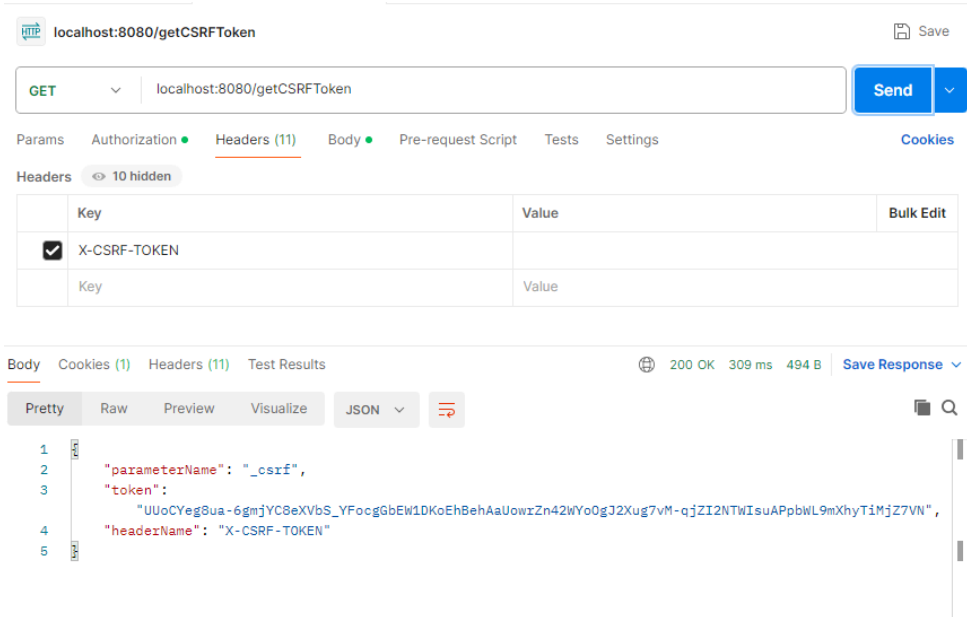
Example :

```
@GetMapping("/getCSRFToken")
public CsrfToken getCsrfToken(HttpServletRequest request) {
    return (CsrfToken) request.getAttribute("_csrf");
}
```

Spring Security

A quick Guide to learn spring security

8. Access it from postman/ browser you will get the csrf token value and use in the postman to save the data.

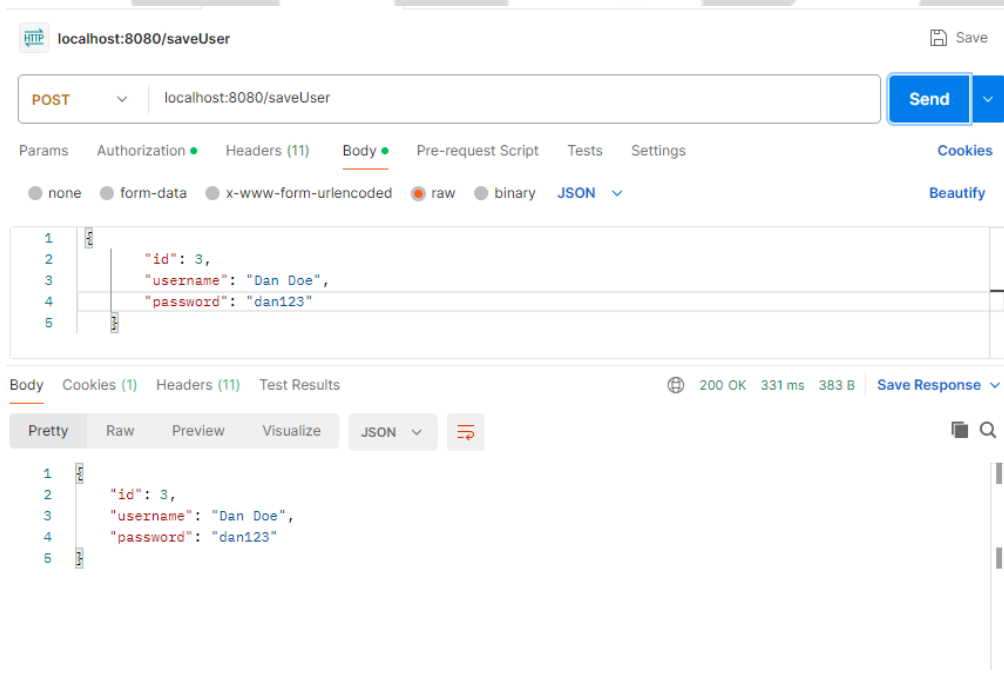


9. To send the CSRF token we have go to header in postman and provide the below syntax

Syntax :

X-CSRF-TOKEN : value

10. Once we use the CSRF token it will perform the POST, PUT and DELETE operation



11. Here everytime we have to get the CSRF token which is not a efficient way to work with it we can generate the session id everytime we visit the site.

Spring Security

A quick Guide to learn spring security

Configuring Spring Security :

1. To configure security in spring we have to create a class to create beans.
2. After creating a class declare the class using `@Configuration`.
3. Here to control the flow of spring security instead of following the default flow we use another annotation `@EnableWebSecurity`.
4. Now after doing all that we have to create a bean for spring security filter. We are having a class `SecurityFilterChain`.
5. But to create it we need to ask the `HttpSecurity` to build the object for `SecurityFilterChain`.

Syntax :

```
@Configuration
@EnableWebSecurity
Class className{
    @Bean
    Public SecurityFilterChain methodName(HttpSecurity hsObj){
        //filters here
        Return hsObj.build();
    }
}
```

Example :

```
@Configuration
@EnableWebSecurity
public class SecurityConfiguration {
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws
Exception {

        return httpSecurity.build();
    }
}
```

6. If we didn't write the filters then we will not be able to implement the security in our project.
7. Configuring the filters.

a. Csrf :

Disabling the csrf can be done in two ways.

1. Using lambda expression
2. Using without lambda expression

1. Using lambda Expression :

To do that we have to use the arrow function.

Syntax :

```
hsObj.csrf( param -> param.disable());
```

Example :

```
//Disables the CSRF in the application
httpSecurity.csrf(customizer -> customizer.disable());
```

b. Activating authentication :

To activate the authorization we have to use the `hsObject` and use lambda expression.

Syntax :

```
hsObj.authorizeHttpRequest(param -> param.anyRequest().authenticated());
```

Example :

```
//enables the authentication
```

Spring Security

A quick Guide to learn spring security

```
httpSecurity.authorizeHttpRequests(request ->
request.anyRequest().authenticated());
```

NOTE : Just by providing the `HttpSecurity authorizeHttpRequest` doesn't provide the login form, but it will keep you from visiting the page.

c. Login Form :

Syntax :

```
hsObj.formLogin( Customizer.withDefaults());
```

Example :

```
//enables loginform
httpSecurity.formLogin(Customizer.withDefaults());
```

NOTE : It will not provide access to the rest apis, hence we have to use another method to access the data.

Syntax :

```
hsObj.httpBasic(Customizer.withDefaults());
```

Example :

```
//enables loginform in REST APIs (postman)
httpSecurity.httpBasic(Customizer.withDefaults());
```

d. Making session stateless :

In the csrf token we discussed that creating the session id will eliminate the csrf attack. To make it happen we have to enable it.

Syntax :

```
hsObj.sessionManagement( session ->
session.sessionCreationPolicy(sessionCreationPolicy.STATELESS));
```

Example :

```
//enables new session id generation on each new resource
httpSecurity.sessionManagement(session -> session.sessionCreationPolicy(
    SessionCreationPolicy.STATELESS
));
```

NOTE : When we are making a `httpSession stateless` it will consider each new resource generation as a new session hence we have to login multiple times, just disable the login form in browser.

NOTE : Make sure we are mentioning the `componentScan` annotation in the application end to recognize the bean we are creating.

NOTE : We can even use the chaining method here as they all use the same object then we can use it.

Example :

```
httpSecurity.csrf(customizer -> customizer.disable())
    .authorizeHttpRequests(request -> request.anyRequest().authenticated())
    .formLogin(Customizer.withDefaults())
    .httpBasic(Customizer.withDefaults())
    .sessionManagement(session -> session.sessionCreationPolicy(
        SessionCreationPolicy.STATELESS
    ));
```


Spring Security

A quick Guide to learn spring security

Setting User from Database :

Generally we are using the username and password in spring which can be only one, but when we are working in real time we will have multiple users. Hence we use the UserDetailsService to work with the user credentials.

Steps to create the bean for userDetailsService :

1. Create a bean in the spring security configuration file.
2. Use the bean annotation to create the bean but here UserDetailsService is a interface hence if we need to create a object then we need the class.

Syntax :

```
@Bean
Public UserDetailsService methodName(){
    //configuration here
}
```

3. We have to use the class InMemoryUserDetailsManager to create the object.
4. We need to pass an parameter to create the user as InMemoryUserDetailsManager just manage the users but will not create one.
5. To create we have to use the UserDetails interface, to create an object to the interface we have to use the User class which belongs to spring.

Syntax :

```
UserDetails userDetailsObj = User.withDefaultPasswordEncoder()
                                .username("user name here")
                                .password("password here")
                                .roles("user role can be provided here")
                                .build();
```

6. Then pass the userDetailsObj within the constructor of the InMemoryUserDetailsManager

Syntax :

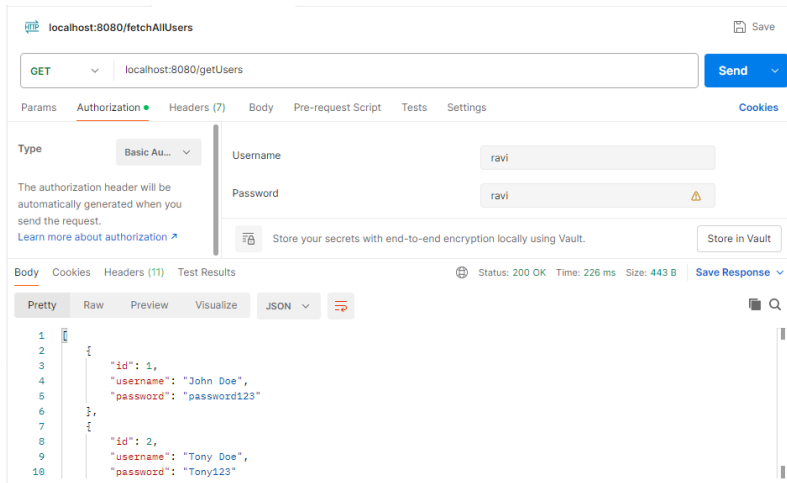
```
Return new InMemoryUserDetailsManager(userDetailsObj);
```

Example :

```
@Bean
public UserDetailsService userDetails() {
    UserDetails user1 = User.withDefaultPasswordEncoder()
                            .username("Ravi")
                            .password("ravi")
                            .roles("admin", "trainer")
                            .build();
    return new InMemoryUserDetailsManager(user1);
}
```

Spring Security

A quick Guide to learn spring security



- But we are not connecting with the database at all now we need to connect with the database to work with the users. In general we will have the un-authenticated data will be sent to authentication provider to authenticate the data that data is called as object.
- Now we need to customize the authentication provider. For database we are having DaoAuthenticationProvider.

Syntax :

```
//class implementing the UserDetailsService
@Autowired
Private customClass userDetailsService;
@Bean
Public AuthenticationProvider authenticationProvider(){
    DaoAuthenticationProvider daoProObj = new DaoAuthenticationProvider();
    daoProObj.setPasswordEncoder(NoOpPasswordEncoder.getInstance());
    daoProObj.setUserDetailsService(userDetailsService);

    Return daoProObj;
}
```

Example :

```
@Autowired
DatabaseUserDetailsService userDetailsService;

@Bean
public AuthenticationProvider getAuthentication() {
    DaoAuthenticationProvider daoAuthentication = new DaoAuthenticationProvider();

    daoAuthentication.setPasswordEncoder(NoOpPasswordEncoder.getInstance());
    daoAuthentication.setUserDetailsService(userDetailsService);

    return daoAuthentication;
}
```

Spring Security

A quick Guide to learn spring security

9. We have to connect with the database and we have to provide it's object to the custom class and add unimplemented method(loadUserByUsername).

DatabaseUserDetailsService class :

```
@Service
public class DatabaseUserDetailsService implements UserDetailsService{

    @Autowired
    UsersDao usersDao;

    @Override
    public UserDetails loadUserByUsername(String username) throws
    UsernameNotFoundException {
        Users user = usersDao.getByUsername(username);
        System.out.println("Inside the loadUserByUsername method");
        if(user == null) {
            System.err.println("User not found");
            throw new UsernameNotFoundException("User not found");
        }

        System.err.println(user);
        //custom class which will create object for the UserDetails
        return new UserPrincipal(user);
    }
}
```

UserDao Example :

```
@Repository
public interface UsersDao extends JpaRepository<Users, Integer>{
    Users getByUsername(String username);
}
```

10. Then we need to fetch the data from database and need to pass to another customCreated class which will create the UserDetails Object by implementing it.
11. There we need to implement some methods which belongs to the UserDetails.

Users table in Database :

id	name	password	role
1	ravi	ravi@ravi	admin
2	test1	test@password1	user
NULL	NULL	NULL	NULL

NOTE : When we are referring to the current user we mention them as userprincipal

UserPrincipal class :

```
public class UserPrincipal implements UserDetails {

    private Users user;
```

Spring Security

A quick Guide to learn spring security

```
public UserPrincipal(Users user){
    this.user = user;
}
@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    ArrayList<GrantedAuthority> authorities = new ArrayList<GrantedAuthority>();

    authorities.add(new SimpleGrantedAuthority(user.getRole()));

    return authorities;
}
@Override
public String getPassword() {
    // TODO Auto-generated method stub
    return user.getPassword();
}
@Override
public String getUsername() {
    // TODO Auto-generated method stub
    return user.getUsername();
}
}
```

12. Create an entity class to match with the database table. We call them as models in java, hence we have to place them in the model package.

Users class Example :

```
@Entity
public class Users {
    @Id
    private Integer id;
    private String name;
    private String password;
    private String role;

    public Users() {
        super();
    }
    public Users(Integer id, String username, String password, String role) {
        super();
        this.id = id;
        this.name = username;
        this.password = password;
        this.role = role;
    }
    public Integer getId() {
        return id;
    }
    public void setId(Integer id) {
        this.id = id;
    }
}
```

Spring Security

A quick Guide to learn spring security

```
}  
public String getUsername() {  
    return name;  
}  
public void setUsername(String username) {  
    this.name = username;  
}  
public String getPassword() {  
    return password;  
}  
public void setPassword(String password) {  
    this.password = password;  
}  
@Override  
public String toString() {  
    return "Users [id=" + id + ", username=" + name + ", password=" + password + ",  
role=" + role + "];"  
}  
public String getRole() {  
    return role;  
}  
public void setRole(String role) {  
    this.role = role;  
}  
}
```

Encrypting password :

Password is the most sensitive information as most of the users will use the same password for multiple accounts. So we can't store the plain text into our database as there is a chance that it can be vulnerable to others. Hence we use the encryption algos to convert our plain text to the hash code.

In market we are having lot of algos available, here we will use the bcrypt which will generate the plain text to hashcode and it can't be reverted.

Bcrypt password Generator will perform the a set of rounds on a generated hashcode with n^{10} where n will be the no.of rounds. Most optimized rounds are 10 or 12.

In spring security we will have the bcrypt library by default.

Using Bcrypt in spring project :

1. To use the Bcrypt algo in the project create an object for the BCryptPasswordEncoder.
2. In constructor of the BCryptPasswordEncoder we can pass the no.of rounds

Syntax :

```
BCryptPasswordEncoder bcryptPassword = new BCryptPasswordEncoder(rounds);
```

```
Public returnType methodName(params){  
    bcryptPassword.encode(password);
```

```
    Return value;
```

```
}
```

Example :

Spring Security

A quick Guide to learn spring security

```
public Users registerUser(Users user) {  
    // TODO Auto-generated method stub  
    BCryptPasswordEncoder passwordEncoder = new  
BCryptPasswordEncoder(12);  
    user.setPassword(passwordEncoder.encode(user.getPassword()));  
    return userDao.save(user);  
}
```

3. But again here we are just using the bcryptObj to encrypt the password and we are not authenticating it.
4. To authenticate it we have to change the NoOpPasswordEncoder to just pass the BCryptPasswordEncoder object along with the no.of rounds matching the encoding part.

Syntax :

```
daoAuthenticateObj.setPasswordEncoder(new BCryptPasswordEncoder(rounds));
```

Example :

```
@Bean  
public AuthenticationProvider getAuthentication() {  
    DaoAuthenticationProvider daoAuthentication = new  
DaoAuthenticationProvider();  
    //not encoding the password which used in the earlier stage  
    daoAuthentication.setPasswordEncoder(NoOpPasswordEncoder.getInstance());  
    //daoAuthentication which using the BCryptPasswordEncoder  
    daoAuthentication.setPasswordEncoder(new BCryptPasswordEncoder(12));  
    daoAuthentication.setUserDetailsService(userDetailsService);  
  
    return daoAuthentication;  
}
```

Table data in database :

id	name	password	role	username
1	ravi	ravi@ravi	admin	NULL
2	test1	test@password1	user	NULL
3	hari	\$2a\$12\$FmsUdnpygJrAIt8shVM9CeSMJKhCfQY...	admin	NULL
4	tony	\$2a\$12\$mEtUX5ICL4.NS/V.Xer/rueSealS1ZuIH...	user	NULL
NULL	NULL	NULL	NULL	NULL

Using of JWT :

JWT mean Json Web Token which is used to validate between server and user. To validate we will provide a encoded form of the data to the user during the time of login and it will have the issued and expiry duration for the token in it.

Spring Security

A quick Guide to learn spring security

Encoded PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gR991IiwiaWF0IjoxNTE2MjM5MDIyLCJleHAiOjE1MTY0MzkwMjJ9.SfgQKTwQV6yPybCbGYSEkf9VvDemvILnEkeIGDG10
```

Decoded EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{  "alg": "HS256",  "typ": "JWT"}
```

PAYLOAD: DATA

```
{  "sub": "1234567890",  "name": "John Doe",  "lat": 1516039822,  "exp": 1516439822}
```

VERIFY SIGNATURE

```
HMACSHA256(  base64UrlEncode(header) + ".",  base64UrlEncode(payload),  your-256-bit-secret)
```

☐ secret base64 encoded

Signature Verified

SHARE JWT

JWT token is having 3 parts :

1. Header
2. Payload
3. Verify signature

Implementing JWT in spring project :

1. In our spring project we have to add some dependencies to work with JWT.
2. Goto Maven Repo -> search for JJWT API, jjwt jackson and JJWT :: Impl
3. Once we are done adding the dependencies we have to goto our securityConfiguration class. Create a bean here for AuthenticationManager interface.

Syntax :

```
@Bean
Public AuthenticationManager authenticationManager(AuthenticationConfiguration param1){
    Return param1.getAuthenticationManager();
}
```

Example :

```
@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration
authConfiguration) throws Exception {
    return authConfiguration.getAuthenticationManager();
}
```

4. We need to have some files to work with the JWT
 - a. **JwtUtils/Service** - validate and generate,... Jwt
 - b. **AuthTokenFilter** - check for the valid JWT token(Custom Filter)
 - c. **AuthEntryPointJwt** - Will handle the unauthorized tokens
5. Here we have to manage the authentication for the requests, to do that we will change some configurations in the springsecurity configuration file.

Syntax to skip the authentication :

```
http.csrf(customizer -> customizer.disable()).authorizeHttpRequest(
Request ->
Request. requestMatchers("...patterns", "request-pattern1", "request-pattern2").permitAll()
```

Spring Security

A quick Guide to learn spring security

```
        .anyRequest().authenticated()
    ).
    httpBasic(customizer -> withDefault());
```

Example :

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws
Exception{

    httpSecurity.csrf(customizer -> customizer.disable())
    .authorizeHttpRequests(
        request -> request.requestMatchers("/register", "/loginUser",
"/home")
    .permitAll()
    .anyRequest().authenticated()
    .formLogin(Customizer.withDefaults())
    .httpBasic(Customizer.withDefaults())
    );

    return httpSecurity.build();
}
```

6. Here we have to make sure we are checking the user before logging in to the website. To Do that we need to use the authenticationManager object in the service class to verify the data.
7. In service class we have to authenticate the username and password. To authenticate we are having the Authenticate interface which we need to create a class.

Syntax :

```
@Autowired
Private AuthenticateManager authManagerObj;
Public returnType verifyUser(UserClass obj){
    Authentication authenticate = authManagerObj.authenticate(
        new UsernamePasswordAuthenticationToken(
            obj.getUserName(),
            obj.getUserPassword
        )
    );

    if(authenticate.isAuthenticated()){
        Return "Success";
    }
}
```

Example :

```
public String verifyUser(Users user) {
    Authentication auth = authManager.authenticate
    (new
    UsernamePasswordAuthenticationToken(user.getUsername(), user.getPassword()));
    if(auth.isAuthenticated())
        return "Success";
    return "Failed";
}
```


Spring Security

A quick Guide to learn spring security

8. We are not creating any token here but we need to manually create a token and need to send to the user. To do that we have to create a separate class for the JWT to create a token.
9. To Generate a token we need some attributes to create. To do that we have to create a map to store all of the values which are needed to create a token.

Syntax :

```
Map<String, Object> mapObj = new HashMap();
```

10. To create a token use the class `Jwt` which is used to generate the claims(token info) here we have to follow the below syntax.

Syntax :

```
Return Jwts.builder().claims()
    .add(mapObj)
    .subject("Add subject")//use username here
    .issuedAt(new Date(System.currentTimeMillis()))
    .expiration(new Date(System.currentTimeMillis() + secs * mins * mins)
    .and()
    .signWith(key) //key has to be used here
    .compact();
```

11. To generate a key we have to use the `KeyGenerator` which belongs to `java.crypto` package. Easy way of doing it is we can generate a key when we are creating an object for `JwtService` class

Syntax :

```
String key;
Public class Name(){
    KeyGenerator keyObj = new KeyGenerator.getInstance("AlgoName");
    SecretKey skObj = keyObj.generateKey(); //generate key
    Key = Base64.getEncoder().encodeToString(skObj.getEncoded());
}
```

Example :

```
private SecretKey key;
private Map<String, Object> claimsMap = new HashMap<String, Object>();
public JwtService() {
    try {
        KeyGenerator keyGen = KeyGenerator.getInstance("HmacSHA256");
        key = keyGen.generateKey();
    } catch (NoSuchAlgorithmException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}

public String generateToken(String username) {
    // TODO Auto-generated method stub
    return Jwts.builder()
        .claims(claimsMap)
        .subject(username)
        .issuedAt(new Date(System.currentTimeMillis()))
        .expiration(new Date(System.currentTimeMillis() + 60 * 60 * 1000)) // Set expiration to 1
        hour
        .signWith(key)
        .compact();
}
```

Spring Security

A quick Guide to learn spring security

To Verify the token at the Server end :

1. Here the trick is to make the user login only for once and we provide a token to the user and every time if he is asking for new resource we must be able to provide him the data by verifying the token
2. We call it as Bearer token and by default we use the UserPasswordAuthentication Filter here we need to change the order of filters, 1st we need to add the Token authenticator and second is UserPasswordAuthenticationFilter
3. To do that we are having a methods addFilter which we need to mention when we need to place the filter at.

Syntax :

```
.addFilterBefore(filter1,filter2);
```

4. If we have to create a custom filter class then place them in filters package.
5. Once we created a custom filter class we have to extend the class to make it behave like filter OncePerRequestFilter, Add an unimplemented method doFilterInternal

Syntax :

```
Class className extends OncePerRequestFilter{  
    //code here  
}
```

Example :

```
public class JwtFilter extends OncePerRequestFilter{  
    @Override  
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,  
        FilterChain filterChain)  
        throws ServletException, IOException {  
        //code here to handle exception  
    }  
}
```

6. In that we will get a Bearer secretKey... We have to remove the Bearer from the key when we are passing the request.

NOTE: The Header will have the Bearer when we are passing a valid JWT token.

7. Here we are going to verify the header, username and jwtToken. To do that we have to get the token from the request by using the ServletRequest.

Syntax :

```
String header = httpServletRequest.getHeader(AUTHORIZATION)  
if(header != null && header.startsWith("Bearer")){  
    //code  
}
```

8. Now to get the token by removing the Bearer we have to use the substring method to do that generally we will have "Bearer token" if we count it will start from 0 to 7 we need to remove so we need to start from 7 to get the token.

Syntax :

```
Token = header.substring(7);
```

9. After getting the token we have to extract the username from the token, to do that we need to use the JwtService to get the username.

Syntax :

```
Username = jwtService.extractUsername(jwtToken);
```

Example :

```
if(authHeader != null && authHeader.startsWith("Bearer")) {  
    //seperating the token with Bearer word  
    jwtToken = authHeader.substring(7);  
    //extracting the username from jwtToken  
}
```

Spring Security

A quick Guide to learn spring security

```
        username = jwtService.extractUsername(jwtToken);
    }
```

10. After extracting the username check if the username is valid and also the request is not authenticated then only we need to create a object for the jwtFilter and need to validate and to verify user we need to verify it from database, hence we need to use the UserDetails Service as well.

Syntax :

```
    if(jwtService.validateToken(jwtToken, userDetails){
        //code here
    }
```

NOTE : Everytime if we are using the repeated Autowired in multiple classes it will cause a cyclic redundancy. So that we can use the ApplicationContext object to getBean.

Syntax :

```
    @Autowired
    ApplicationContext context;
```

11. Here also to get the username we are having the DatabaseUserDetailsService to return the userDetails object.

Syntax :

```
    UserDetails userDetailsObj =
    context.getBean(DatabaseUserDetailsService.class).loadUserByUsername(username);
```

Example :

```
        if(username != null &&
            SecurityContextHolder.getContext().getAuthentication() == null) {

            //load user details from database or any other source based on username
            UserDetails userDetails = applicationContext
                .getBean(DatabaseUserDetailsService.class)
                .loadUserByUsername(username);

            //validating the token
            if(jwtService.validateToken(jwtToken, userDetails)) {

            }
        }
    }
```

12. Once the token is validated then we have to carryon to the next filter
UserPasswordAuthenticationFilterToken, to do that we have to pass 3 parameters.

- a. userDetails
- b. Credentials
- c. authorities

Syntax :

```
    UsernamePasswordAuthenticationToken upaToken = new
    UsernamePasswordAuthenticationToken(userDetailsObj, null, userDetails.getAuthorities());
```

Example :

```
        if(jwtService.validateToken(jwtToken, userDetails)) {
            UsernamePasswordAuthenticationToken upaToken =
                new UsernamePasswordAuthenticationToken(
                    userDetails,
                    null,
                    userDetails.getAuthorities()
                );

            //upaToken willnot have any data associated with the request
        }
    }
```

Spring Security

A quick Guide to learn spring security

13. As the upaToken will not know the request we have to set the request object into the token. To do that we have to use the method `setDetails()`. `setDetails` will need a constructor as a argument. We need to pass the constructor for `WebAuthenticationDetailsSource()`.

Syntax :

```
upaToken.setDetails(  
    New WebAuthenticationDetailsSource().buildDetails(requestObj)  
);
```

14. After doing it we have to add the above step that means we verified the authentication and we have to set the authentication.

Syntax :

```
SecurityContextHolder.getContext().setAuthentication(upaToken);
```

Example :

```
//upaToken willnot have any data associated with the request so add request to it  
upaToken.setDetails(new WebAuthenticationDetails(request));  
//now we need to update the authentication as authenticated in securityHolder  
SecurityContextHolder.getContext().setAuthentication(upaToken);
```

15. Once we are done the authentication we can move for the next filter in the filterChain to do that we have use `doFilter()` method.

Syntax :

```
filterChain.doFilter(request, response);
```

Example :

```
//once we are done there we need to move to next filter  
filterChain.doFilter(request, response);
```

16. The complete JwtFilter class will look as shown below.

Example :

```
package com.configuration.Filters;  
import java.io.IOException;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.context.ApplicationContext;  
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;  
import org.springframework.security.core.context.SecurityContextHolder;  
import org.springframework.security.core.userdetails.UserDetails;  
import org.springframework.security.web.authentication.WebAuthenticationDetails;  
import org.springframework.stereotype.Component;  
import org.springframework.web.filter.OncePerRequestFilter;  
import com.configuration.Service.DatabaseUserDetailsService;  
import com.configuration.Service.JwtService;  
import jakarta.servlet.FilterChain;  
import jakarta.servlet.ServletException;  
import jakarta.servlet.http.HttpServletRequest;  
import jakarta.servlet.http.HttpServletResponse;  
//helps to create a bean in spring IOC container  
@Component  
public class JwtFilter extends OncePerRequestFilter{  
  
    //autowiring the jwt service class to utilize the methods of jwt service class  
    @Autowired  
    JwtService jwtService;  
  
    //using it to reduce the cyclic redundancy of userDetails
```

A quick Guide to learn spring security

@Autowired

```
ApplicationContext applicationContext;
```

@Override

```
protected void doFilterInternal(
    HttpServletRequest request,
    HttpServletResponse response,
    FilterChain filterChain)
    throws ServletException, IOException {
```

```
//getting the header from the request
```

 ℓ^*

* sample request

- * POST /api/resource HTTP/1.1

Host: example.com

Authorization: Bearer

[eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjYybmFtZSIsImhhbmQiOiJMTYxNjZOTAYmMiwiZXhwIjoxNjE2MjM0ODIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c](#)

Content-Type: application/json

```
{
  "data": "value"
}
```

* *

```
//getting the token part from the request which is having the Bearer
```

```
final String authHeader = request.getHeader("Authorization");
```

```
String username = null;
```

```
String jwtToken = null;
```

```
//verifying wether the token is valid or not
```

```
if(authHeader != null && authHeader.startsWith("Bearer")) {
```

```
//seperating the token with Bearer word
```

```
jwtToken = authHeader.substring(7);
```

```
//extracting the username from jwtToken
```

```
username = jwtService.extractUsername(jwtToken);
```

}

```
//check if the username is valid or not and also verifying whether the request
```

authenticated or not

 ℓ^*

* The SecurityContextHolder in Spring Security is a core part

* of the framework responsible for storing security-related

* details, primarily the authentication and user information

- * during the lifecycle of a request.

* *

```
if(username != null &&
```

```
SecurityContextHolder.getContext().getAuthentication() == null) {
```

```
//load user details from database or any other source based on
```

username

```
UserDetails userDetails = appContext
```

```
.getBean(DatabaseUserService.class)
```

Spring Security

A quick Guide to learn spring security

```
.loadUserByUsername(username);
//validating the token if it is valid continue to the next filter
if(jwtService.validateToken(jwtToken, userDetails)) {
    UsernamePasswordAuthenticationToken upaToken =
        new
        UsernamePasswordAuthenticationToken(userDetails,null,userDetails.getAuthorities());
//upaToken willnot have any data associated with the request so add request to it
upaToken.setDetails(new WebAuthenticationDetails(request));
//now we need to update the authentication as authenticated in securityHolder
SecurityContextHolder.getContext().setAuthentication(upaToken);
}
}

//once we are done there we need to move to next filter
filterChain.doFilter(request, response);

}
}
```

Verifying username and validating token in JwtService :

1. Here we need to extract the username and expiration to validate the token, we need to get the username from database to check if the username in token is same or not.
2. Also we need to verify whether the token is not expired.
3. To do that we have to create multiple methods each method will do a certain task like extracting the username from token and another one extracting expiration from token.

```
package com.configuration.Service;
import java.security.Key;
import java.security.NoSuchAlgorithmException;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
@Service
public class JwtService {

    //holds the secretkey we are creating initially
    private SecretKey key;

    //will map the details that are needed for the claims
    private Map<String, Object> claimsMap = new HashMap<String, Object>();

    //here we are generating the secretkey during the object creation
```

Spring Security

A quick Guide to learn spring security

```
public JwtService() {
try {
    //KeyGenerator class will get its instance and use hmac SHA algo to generate key
    KeyGenerator keyGen = KeyGenerator.getInstance("hmacSHA256");
    //generate the secretkey
    key = keyGen.generateKey();
} catch (NoSuchAlgorithmException e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
}

}

//returns the secret key
private SecretKey getKey() {
    return key;
}

//will generate the jwt token
public String generateToken(String username) {
    // TODO Auto-generated method stub
    /*
    * Jwts - class for JSON Web Token
    * builder() method initializes a new builder instance for creating a JWT
    * claims() This method allows you to set custom claims in the JWT. - pass map as
arguments
    * subject() his method sets the subject claim of the JWT.- pass the username as
argument
    * issuedAt() method helps us to provide the token issue date
    * expiration() method helps us to provide the token expiration date - pass milliseconds as
argument
    * signWith() method helps us to mention which key has to use to sign - pass secret key as
argument
    * compact() This method finalizes the building process and generates the JWT as a
compact, URL-safe string.
    */
    return Jwts.builder()
        .claims(claimsMap)
        .subject(username)
        .issuedAt(new Date(System.currentTimeMillis()))
        .expiration(new Date(System.currentTimeMillis() + 60 * 60 * 1000))
        // Set expiration to 1 hour in milli seconds
        .signWith(key)
        .compact();
}

//username extraction part
public String extractUsername(String jwtToken) {
    /*
    * Here we are passing the arguments of jwtToken and Claims class method getSubject
    * The Claims::getSubject method is used to access the subject claim from a JWT.
    */
}
```

Spring Security

A quick Guide to learn spring security

```
        return extractClaim(jwtToken, Claims::getSubject);
    }

    //extracts the claim and it will take a function
    /*
    * return type is <T> T which represents Generic type
    * there are some set of generic types which we can use in java.
    * Common Generic Placeholders:
        T stands for Type.
        E stands for Element (used in collections like List<E>, Set<E>).
        K stands for Key (used in Map<K, V>).
        V stands for Value (used in Map<K, V>).
        N stands for Number.
    Why Use Generics?
    Type Safety: Generics allow the compiler to check that the types used are correct, reducing the
    chances of ClassCastException at runtime.
    Code Reusability: You can write more flexible and reusable code since you can apply it to different
    types without needing to rewrite it for each type.
    */
    private <T> T extractClaim(String jwtToken, Function<Claims, T> claimResolver) {
        final Claims claims = extractAllClaims(jwtToken);
        return claimResolver.apply(claims);
    }
    private Claims extractAllClaims(String jwtToken) {
        /*
        * Jwts.parser() - The parser is used to decode and validate JWTs.
        * verifyWith() - This method is used to specify the key that will be used to verify the
signature of the JWT.
        * build() - This finalizes the setup of the parser.
        * parseSignedClaim() - This method takes the JWT as a string (jwtToken) and attempts to
parse it.
        * getPayload() - This method extracts the payload (the claims) from the parsed JWT
        */
        return Jwts.parser()
            .verifyWith(getKey())
            .build()
            .parseSignedClaims(jwtToken)
            .getPayload();
    }
    //token validation
    public boolean validateToken(String jwtToken, UserDetails userDetails) {
        //calling the extractUsername method to get the username from token
        final String userName = extractUsername(jwtToken);
        //validating the username by checking in the database and expiration of the token
        Boolean validated = userName.equals(userDetails.getUsername()) &&
!isTokenExpired(jwtToken);
        return validated;
    }

    //checking if token is expired or not by before date
    private boolean isTokenExpired(String jwtToken) {
```


Spring Security

A quick Guide to learn spring security

```
/*
 * It returns true if the expiration date of the token is before the current date and time,
 indicating that the token is no longer valid.
 It returns false if the token has not yet expired.
 */
return extractExpirationToken(jwtToken).before(new Date());
}
//extracting the expiration token
private Date extractExpirationToken(String jwtToken) {
    //extracting the expiration using the extractClaim method
    return extractClaim(jwtToken, Claims::getExpiration);
}
}
```

Logging through third party apps :

1. We can verify user based on the 3rd party logins like google, github,...
2. To do that we need to use the dependencies mentioned below
 - a. Spring web
 - b. OAuth2Client - will provide the spring security dependencies by default
3. Now we need to create a controller to accept the request and need a SecurityConfig file to configure.
4. But we need the 3rd party authentication part here to do that we are considering the github to verify the user.
5. To do that we have to mention some properties in the application.properties

Syntax :

```
spring.security.oauth2.client.registration.github.client-id=
spring.security.oauth2.client.registration.github.client-secret=
```

6. To get the values for them we have to goto github developer settings create a Click on "New OAuth App" and fill out the following details:
 - **Application Name:** The name of your app.
 - **Homepage URL:** URL of your app (for local development, use <http://localhost:8080>).
 - **Authorization callback URL:** This is the URL GitHub redirects to after authentication. For local development, use <http://localhost:8080/login/oauth2/code/github>.
7. Once we are done filling out the form we can get the client-id and client-secret

Example :

```
spring.security.oauth2.client.registration.github.client-id= Ov23li19Uoltwlvh54y3l
spring.security.oauth2.client.registration.github.client-secret=94f4a8abc734452064a1
```

8. Also we need to enable it in the security configuration as well to do that we have to use oauth2Login() method inside it pass the Customizer.withDefaults. Add that method in the flow chain.

Syntax :

```
oauth2Login(Customizer.withDefaults())
```

Example :

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws Exception{

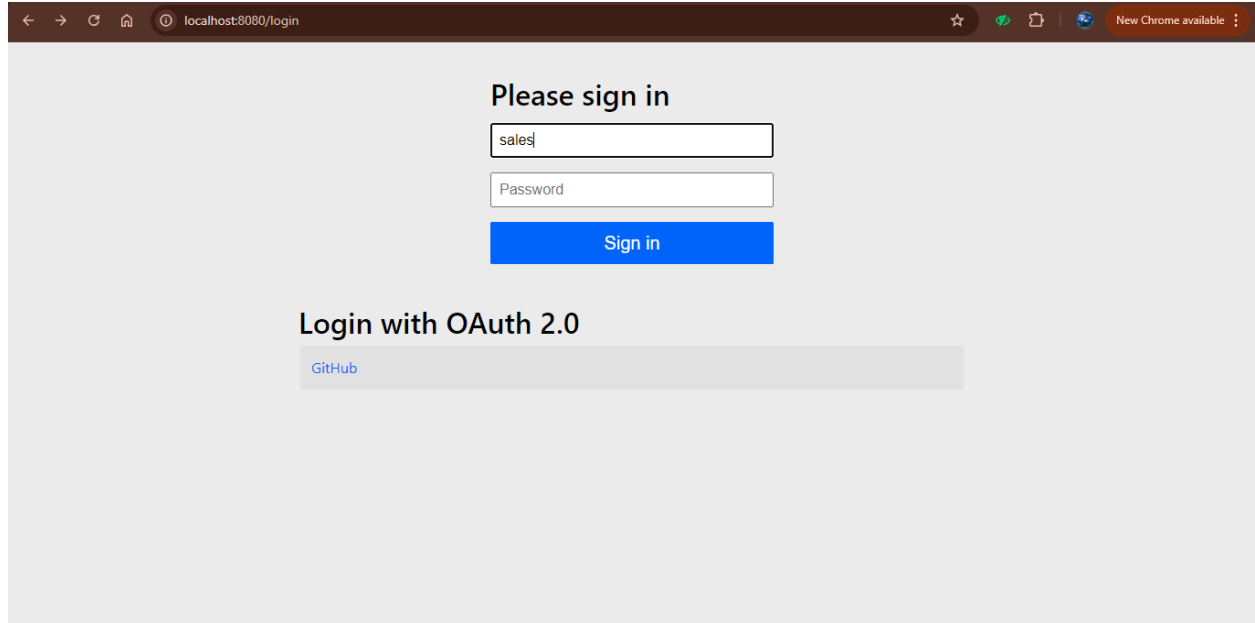
    return httpSecurity.csrf(customizer -> customizer.disable())
        .authorizeHttpRequests(
            request -> request.requestMatchers("/register", "/loginUser",
"/homepage")
                .permitAll()
                .anyRequest().authenticated()
        )
}
```

Spring Security

A quick Guide to learn spring security

```
.oauth2Login(Customizer.withDefaults())  
.formLogin(Customizer.withDefaults())  
.httpBasic(Customizer.withDefaults())  
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)  
.build();  
}
```

Output :



BINARY DEVS
ACADEMY